



Faculty of Engineering
Scripting Language Python
Laboratory Manual

NAME:

ENROLLMENT NUMBER:

SUBJECT NAME:

SUBJECT CODE: **BRANCH:**

SEMESTER: **ACADEMIC YEAR:**



Swaminarayan university
Publication

ISBN Number: 978-81-998196-4-1



सर्व विद्या अधिपतये

Certificate

This is to certify that Mr. / Ms. / Mrs.
having enrollment number of Semester ME /
BE / DIPLOMA in.....has satisfactorily completed his
term working (Subject name) (subject code:.....)
during academic year at Faculty of Engineering, Swaminarayan University.

Place:

Date:

ASHWINI WALAKE
SUBJECT IN-CHARGE

HOD

DR. BHAVNA SONI
DEAN

List of Practical

Sr. No.	Title			Page No.	Date	Sign
1	Install and configure the Python environment. Run basic Python commands to verify the Python environment.					
2	Write a program to read your name, contact number, email, and birthdate and print those details on the screen.					
3	Write a Python program that calculates the area of a circle based on the radius entered by the user.					
4	Write a Python Program that perform all Arithmetic Operation with input() Function					
5	Write a Program to swap two numbers using three variables.					
6	Write a Python program that calculates the area of a circle based on the radius entered by the user.					
7	Demonstrate code that display mark sheet (enter subject wise marks, total marks, percentage)(use type casting)					
8	Write a Program that check maximum value from given two values.					
9	Write a code that checks whether given number is even or odd.					
10	Write a code that check whether given number is Positive, Negative or zero(use elif method)					
11	Write a code that generate mark sheet(refer code:6) and decide grade of students (using elif method)					
12	Write a code that find maximum from given three number using nested if else.					
13	i)Write a code that print 1....10 using for loop ii)Write a code that print 1....10 using while loop					
14	Write a code that print even numbers and give sum of those even number.(using int input() function)					
15	(i)Write a code that finds you factorial of given natural numbers. (ii) Write a code that check that given number is even or odd.					
16	Draw following patterns using python code with the help of nesting of loop.					
	1 1 2 1 2 3 1 2 3 4 1 2 3 4 5	* * * * * * * * * * * * * * *	A BC DEF GHIJ KLMNO			

17	Write a program to read the marks and assign a grade to a student. Grading system: A (≥ 90), B (80-89), C (70-79), D (60-69), E (50-59), F (< 50). (using switch case)			
18	Write a program to perform the below operations on the list: • Create a list • Add/Remove an item to/from a list. • Get the number of elements in the list. • Access elements of the list using the index. • Sort the list. • Reverse the list.			
19	Write a program to eliminate duplicate values in the list.			
20	Write a program to perform below operations on tuple: • Create a tuple with different data types. • Print tuple items. • Convert tuple into a list. • Remove data items from a list. • Convert list into a tuple. • Print tuple items.			
21	Write a program to perform below operations on set: • Create two different sets with the data. • Print set items. • Add/remove items in/from a set. • Perform operations on sets: union, intersection. • Difference, symmetric difference, checks subset of another set.			
22	Write a program to perform below operations on dictionary: • Create a dictionary. • Print dictionary items. • Add/remove key-value pair in/from a dictionary. • Check whether a key exists in a dictionary. • Iterate through a dictionary. • Concatenate multiple dictionaries.			
23	Write a program that is given a dictionary containing the average daily temperature for each day of the week, and prints all the days on which the average temperature was between 40 and 50 degrees.			
24	Write a program that defines functions (mean and deviation), that computes mean and standard deviation of given numbers			
25	Create a user defined module with simple functions for: addition, subtraction, multiplication, division, modulo, square, factorial. Write a program to import the module and access functions defined in the module.			
26	Write a program to check whether a given string is palindrome or not.			

27	Write a program to read a date in the format DD/MM/YYYY and print the same date in MM-DD-YYYY format.			
28	Write a python code which joins all items in a tuple into a string, using Hash character as separator.			
29	Write a python code that checks whether given character in string is in uppercase or not.			
30	Write a python code that checks whether given character in string is in lowercase or not.			
31	Write a Python code which converts upper case into lower and lower case in uppercase.			
32	Write a code in python which give right justification.			
33	Write a Python code which replaces one word with another word.			
34	Write a program to create a text file and write a string to it.			
35	Write a program to Read an entire text file.			
36	Write a program to write a string to a file.			
37	Write a program to write a list of strings to a file.			

Practical-1

AIM: Install and configure the Python environment. Run basic Python commands to verify the Python environment.

Install Python

1. **Download Python:** Go to the Python download page and download the latest version of Python for your operating system (Windows, macOS, or Linux).
2. **Install Python:**
 - **Windows:** Run the downloaded installer. Make sure to check the "Add Python to PATH" checkbox before clicking "Install Now."
 - **macOS:** Open the downloaded .pkg file and follow the instructions.

Linux: Use a package manager like apt for Debian-based systems: `sudo apt-get install python3`, or yum for Red Hat-based systems: `sudo yum install python3`.

Practical-2

AIM : Write a program to read your name, contact number, email, and birthdate and print those details on the screen.

CODE:

```
name = input("Enter your name: ")
contact_number = input("Enter your contact number: ")
email = input("Enter your email: ")
birthdate = input("Enter your birthdate (e.g., YYYY-MM-DD): ")
print("\nUser Details:")
print("Name:", name)
print("Contact Number:", contact_number)
print("Email:", email)
print("Birthdate:", birthdate)
```

Practical-3

AIM Write a Python program that calculates the area of a circle based on the radius entered by the user.

CODE:

```
import math

radius = float(input("Enter the radius of the circle: "))

area = math.pi * (radius ** 2)

print (f"The area of the circle with radius {radius} is: {area:.2f}")
```

Practical-4

AIM : Write a Python Program that perform all Arithmetic Operation with input()
Function

CODE:

```
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
addition = num1 + num2
subtraction = num1 - num2
multiplication = num1 * num2
division = num1 / num2 if num2 != 0 else "Undefined (cannot divide by zero)"
modulus = num1 % num2 if num2 != 0 else "Undefined (cannot find modulus by zero)"
print(f"\nResults:")
print(f"Addition: {num1} + {num2} = {addition}")
print(f"Subtraction: {num1} - {num2} = {subtraction}")
print(f"Multiplication: {num1} * {num2} = {multiplication}")
print(f"Division: {num1} / {num2} = {division}")
print(f"Modulus: {num1} % {num2} = {modulus}")
```

Practical-5

AIM : Write a Program to swap two numbers using three variables.

CODE:

```
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
print(f"\nBefore swapping: num1 = {num1}, num2 = {num2}")
temp = num1
num1 = num2
num2 = temp
print(f"After swapping: num1 = {num1}, num2 = {num2}")
```

Practical-6

AIM : Write a Python program that calculates the area of a circle based on the radius entered by the user.

CODE:

```
import math
radius = float(input("Enter the radius of the circle: "))
area = math.pi * (radius ** 2)
print(f"The area of the circle with radius {radius} is: {area:.2f}")
```

Practical-7

AIM : Demonstrate code that display mark sheet (enter subject wise marks, total marks, percentage)(use type casting)

CODE:

```
subject1 = float(input("Enter marks for Subject 1: "))
subject2 = float(input("Enter marks for Subject 2: "))
subject3 = float(input("Enter marks for Subject 3: "))
subject4 = float(input("Enter marks for Subject 4: "))
subject5 = float(input("Enter marks for Subject 5: "))
total_marks = subject1 + subject2 + subject3 + subject4 + subject5
percentage = (total_marks / 500) * 100 # Assuming each subject is out of 100 marks
print("\nMark Sheet:")
print(f"Subject 1: {subject1}")
print(f"Subject 2: {subject2}")
print(f"Subject 3: {subject3}")
print(f"Subject 4: {subject4}")
print(f"Subject 5: {subject5}")
print(f"Total Marks: {total_marks}")
print(f"Percentage: {percentage:.2f}%")
```

Practical-8

AIM : Write a Program that check maximum value from given two values.

CODE:

```
value1 = float(input("Enter the first value: "))
value2 = float(input("Enter the second value: "))
if value1 > value2:
    max_value = value1
else:
    max_value = value2
print(f"The maximum value between {value1} and {value2} is: {max_value}")
```

Practical-9

AIM : Write a code that checks whether given number is even or odd.

CODE:

```
number = int(input("Enter a number: "))
    if number % 2 == 0:
        print(f"{number} is even.")
    else:
        print(f"{number} is odd.")
```

Practical-10

AIM: Write a code that check whether given number is Positive, Negative or zero(use elif method)

CODE:

```
number = float(input("Enter a number: "))
if number > 0:
    print(f"{number} is positive.")
elif number < 0:
    print(f"{number} is negative.")
else:
    print(f"{number} is zero.")
```

Practical-11

AIM : Write a code that generate mark sheet(refer code:6) and decide grade of students (using elif method)

CODE:

```
def calculate_grade(percentage):
    if percentage >= 90:
        return "A+"
    elif percentage >= 80:
        return "A"
    elif percentage >= 70:
        return "B+"
    elif percentage >= 60:
        return "B"
    elif percentage >= 50:
        return "C"
    elif percentage >= 40:
        return "D"
    else:
        return "F"

def display_mark_sheet():
    # Prompt the user to enter marks for five subjects
    subject1 = float(input("Enter marks for Subject 1: "))
    subject2 = float(input("Enter marks for Subject 2: "))
    subject3 = float(input("Enter marks for Subject 3: "))
    subject4 = float(input("Enter marks for Subject 4: "))
    subject5 = float(input("Enter marks for Subject 5: "))

    total_marks = subject1 + subject2 + subject3 + subject4 + subject5

    percentage = (total_marks / 500) * 100 # Assuming each subject is out of 100 marks

    grade = calculate_grade(percentage)

    print("\nMark Sheet:")
    print(f"Subject 1: {subject1}")
    print(f"Subject 2: {subject2}")
    print(f"Subject 3: {subject3}")
    print(f"Subject 4: {subject4}")
    print(f"Subject 5: {subject5}")
    print(f"Total Marks: {total_marks}
```

Practical-12

AIM: Write a code that find maximum from given three number using nested if else.

CODE:

```
number1 = float(input("Enter the first number: "))
number2 = float(input("Enter the second number: "))
number3 = float(input("Enter the third number: "))

if num1 >= num2:
    if num1 >= num3:
        max_num = num1
    else:
        max_num = num3
else:
    if num2 >= num3:
        max_num = num2
    else:
        max_num = num3
return max_num

# Print the maximum number
print(f"The maximum number among {number1}, {number2}, and {number3} is: {max_number}")
```

Practical-13

AIM:

- I) Write a code that print 1....10 using for loop**

```
for number in range(1, 11):  
    print(number)
```

- II) Write a code that print 1....10 using while loop**

```
number = 1  
while number <= 10:  
    print(number)  
number += 1
```

Practical-14

AIM: Write a code that print even numbers and give sum of those even number.(using int input() function)

CODE:

```
# Prompt the user to enter the start and end of the range
start = int(input("Enter the start of the range: "))
end = int(input("Enter the end of the range: "))

even_sum = 0

print("\nEven numbers in the range:")
for number in range(start, end + 1):
    if number % 2 == 0:
        print(number)
        even_sum += number
print(f"\nSum of even numbers in the range {start} to {end} is: {even_sum}")
```

Practical-15

AIM: (i) Write a code that finds you factorial of given natural numbers.

CODE:

```
def factorial(n):
    if n == 0:
        return 1
    else:
        return n * factorial(n - 1)
number = int(input("Enter a natural number to find its factorial: "))
if number < 0:
    print("Factorial is not defined for negative numbers.")
else:
    result = factorial(number)
    print(f"The factorial of {number} is: {result}")
```

(ii) Write a code that check that given number is even or odd

```
number = int(input("Enter a number to check if it is even or odd: "))
```

```
if number % 2 == 0:
    print(f"{number} is even.")
else:
    print(f"{number} is odd.")
```

Practical-16

AIM: Write a program to read the marks and assign a grade to a student. Grading system: A (≥ 90), B (80-89), C (70-79), D (60-69), E (50-59), F (< 50). (using switch case)

CODE:

Python program to assign grades using match-case (Python 3.10+)

```
marks = int(input("Enter student marks: "))
```

```
match marks // 10:
```

```
    case 10 | 9:
```

```
        grade = "A"
```

```
    case 8:
```

```
        grade = "B"
```

```
    case 7:
```

```
        grade = "C"
```

```
    case 6:
```

```
        grade = "D"
```

```
    case 5:
```

```
        grade = "E"
```

```
    case _:
```

```
        grade = "F"
```

```
print("Grade =", grade)
```

Practical-17

AIM: Write a program to perform the below operations on the list:

- **Create a list**
- **Add/Remove an item to/from a list.**
- **Get the number of elements in the list.**
- **Access elements of the list using the index.**
- **Sort the list.**
- **Reverse the list.**

CODE:

Create a list

```
my_list = []
```

Add items to the list

```
my_list.append(1)
```

```
my_list.append(2)
```

```
my_list.append(3)
```

```
my_list.append(4)
```

```
my_list.append(5)
```

```
print("List after adding items:", my_list)
```

Remove an item from the list

```
my_list.remove(3) # removes the first occurrence of 3
```

```
print("List after removing 3:", my_list)
```

Get the number of elements in the list

```
num_elements = len(my_list)
```

```
print("Number of elements in the list:", num_elements)
```

Access elements of the list using the index

```
first_element = my_list[0]
```

```
print("First element:", first_element)
```

```
last_element = my_list[-1]
```

```
print("Last element:", last_element)
```

Sort the list

```
my_list.sort()
```

```
print("List after sorting:", my_list)
```

Reverse the list

```
my_list.reverse()
```

```
print("List after reversing:", my_list)
```

Practical-18

AIM: Write a program to eliminate duplicate values in the list.

CODE:

```
def eliminate_duplicates(input_list):  
    # Convert the list to a set to remove duplicates, then convert it back to a list  
    unique_list = list(set(input_list))  
    return unique_list  
  
# Example usage  
input_list = [1, 2, 2, 3, 4, 4, 5, 6, 6, 7, 8, 9, 9, 10]  
print("Original list:", input_list)  
  
unique_list = eliminate_duplicates(input_list)  
print("List after removing duplicates:", unique_list)
```

Practical-19

AIM: Write a program to perform below operations on tuple:

- **Create a tuple with different data types.**
- **Print tuple items.**
- **Convert tuple into a list.**
- **Remove data items from a list.**
- **Convert list into a tuple.**
- **Print tuple items.**

CODE:

```
# Create a tuple with different data types
```

```
my_tuple = (1, "hello", 3.14, True, [5, 6, 7])
```

```
# Print tuple items
```

```
print("Original tuple:", my_tuple)
```

```
# Convert tuple into a list
```

```
my_list = list(my_tuple)
```

```
print("Converted to list:", my_list)
```

```
# Remove data items from the list (let's remove the integer and boolean for example)
```

```
my_list.remove(1)
```

```
my_list.remove(True)
```

```
print("List after removing items:", my_list)
```

```
# Convert the list back into a tuple
```

```
my_tuple = tuple(my_list)
```

```
print("Converted back to tuple:", my_tuple)
```

```
# Print tuple items
```

```
print("Final tuple items:", my_tuple)
```

Practical-20

AIM: Write a program to perform below operations on set:

- Create two different sets with the data.
- Print set items.
- Add/remove items in/from a set.
- Perform operations on sets: union, intersection.
- Difference, symmetric difference, checks subset of another set.

CODE:

```
# Create two different sets with data
```

```
set1 = {1, 2, 3, 4, 5}
```

```
set2 = {4, 5, 6, 7, 8}
```

```
# Print set items
```

```
print("Set 1 items:", set1)
```

```
print("Set 2 items:", set2)
```

```
# Add an item to set1
```

```
set1.add(6)
```

```
print("Set 1 after adding 6:", set1)
```

```
# Remove an item from set2
```

```
set2.remove(6) # Use discard() if you're unsure if the item is in the set
```

```
print("Set 2 after removing 6:", set2)
```

```
# Union of sets
```

```
union_set = set1.union(set2)
```

```
print("Union of Set 1 and Set 2:", union_set)
```

```
# Intersection of sets
```

```
intersection_set = set1.intersection(set2)
```

```
print("Intersection of Set 1 and Set 2:", intersection_set)
```

```
# Difference of sets
```

```
difference_set = set1.difference(set2)
```

```
print("Difference of Set 1 and Set 2 (Set 1 - Set 2):", difference_set)
```

```
# Symmetric difference of sets
```

```
symmetric_difference_set = set1.symmetric_difference(set2)
```

```
print("Symmetric Difference of Set 1 and Set 2:", symmetric_difference_set)
```

```
# Check if set1 is a subset of set2
```

```
is_subset = set1.issubset(set2)
```

```
print("Is Set 1 a subset of Set 2?", is_subset)
```

```
# Check if set2 is a subset of set1
```

```
is_subset = set2.issubset(set1)
```

```
print("Is Set 2 a subset of Set 1?", is_subset)
```

Practical-21

AIM: Write a program to perform below operations on dictionary:

- **Create a dictionary.**
- **Print dictionary items.**
- **Add/remove key-value pair in/from a dictionary.**
- **Check whether a key exists in a dictionary.**
- **Iterate through a dictionary.**
- **Concatenate multiple dictionaries.**

CODE:

```
# Create a dictionary
my_dict = {
    'name': 'John',
    'age': 25,
    'city': 'New York'
}

# Print dictionary items
print("Original dictionary:", my_dict)

# Add a key-value pair to the dictionary
my_dict['email'] = 'john@example.com'
print("Dictionary after adding a key-value pair:", my_dict)

# Remove a key-value pair from the dictionary
del my_dict['age']
print("Dictionary after removing a key-value pair:", my_dict)

# Check whether a key exists in the dictionary
key_to_check = 'city'
if key_to_check in my_dict:
    print(f"Key '{key_to_check}' exists in the dictionary.")
else:
    print(f"Key '{key_to_check}' does not exist in the dictionary.")

# Iterate through the dictionary
print("Iterating through the dictionary:")
for key, value in my_dict.items():
    print(f"{key}: {value}")

# Concatenate multiple dictionaries
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
dict3 = {'e': 5}

# Using the update() method to concatenate dictionaries
concatenated_dict = dict1.copy()
concatenated_dict.update(dict2)
concatenated_dict.update(dict3)
print("Concatenated dictionary:", concatenated_dict)
```

Practical-22

AIM: Write a program that is given a dictionary containing the average daily temperature for each day of the week, and prints all the days on which the average temperature was between 40 and 50 degrees.

CODE:

```
# Create a dictionary
my_dict = {
    'name': 'John',
    'age': 25,
    'city': 'New York'
}

# Print dictionary items
print("Original dictionary:", my_dict)

# Add a key-value pair to the dictionary
my_dict['email'] = 'john@example.com'
print("Dictionary after adding a key-value pair:", my_dict)

# Remove a key-value pair from the dictionary
del my_dict['age']
print("Dictionary after removing a key-value pair:", my_dict)

# Check whether a key exists in the dictionary
key_to_check = 'city'
if key_to_check in my_dict:
    print(f"Key '{key_to_check}' exists in the dictionary.")
else:
    print(f"Key '{key_to_check}' does not exist in the dictionary.")

# Iterate through the dictionary
print("Iterating through the dictionary:")
for key, value in my_dict.items():
    print(f"{key}: {value}")

# Concatenate multiple dictionaries
dict1 = {'a': 1, 'b': 2}
dict2 = {'c': 3, 'd': 4}
dict3 = {'e': 5}

# Using the update() method to concatenate dictionaries
concatenated_dict = dict1.copy()
concatenated_dict.update(dict2)
concatenated_dict.update(dict3)
print("Concatenated dictionary:", concatenated_dict)
```

Practical-23

AIM: Write a program that defines functions (mean and deviation), that computes mean and standard deviation of given numbers

CODE:

```
import math
```

```
def compute_mean(numbers):
```

```
    """
```

```
    Compute the mean (average) of a list of numbers.
```

```
    """
```

```
    total = sum(numbers)
```

```
    count = len(numbers)
```

```
    mean = total / count
```

```
    return mean
```

```
def compute_standard_deviation(numbers):
```

```
    """
```

```
    Compute the standard deviation of a list of numbers.
```

```
    """
```

```
    mean = compute_mean(numbers)
```

```
    variance = sum((x - mean) ** 2 for x in numbers) / len(numbers)
```

```
    standard_deviation = math.sqrt(variance)
```

```
    return standard_deviation
```

```
# Example usage
```

```
numbers = [10, 20, 30, 40, 50]
```

```
mean = compute_mean(numbers)
```

```
standard_deviation = compute_standard_deviation(numbers)
```

```
print(f"Numbers: {numbers}")
```

```
print(f"Mean: {mean}")
```

```
print(f"Standard Deviation: {standard_deviation}")
```

Practical-24

Aim: Create a user defined module with simple functions for: addition, subtraction, multiplication, division, modulo, square, factorial. Write a program to import the module and access functions defined in the module.

CODE:

Step 1: Create a module named `mymath.py`

```
# mymath.py
```

```
def addition(a, b):  
    return a + b
```

```
def subtraction(a, b):  
    return a - b
```

```
def multiplication(a, b):  
    return a * b
```

```
def division(a, b):  
    if b == 0:  
        return "Division by zero is not allowed"  
    return a / b
```

```
def modulo(a, b):  
    return a % b
```

```
def square(a):  
    return a * a
```

```
def factorial(n):  
    if n < 0:  
        return "Factorial not defined for negative numbers"  
    fact = 1  
    for i in range(1, n + 1):  
        fact *= i  
    return fact
```

Step 2: Create another program to import the module (`main.py`)

```
# main.py
```

```
import mymath
```

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))
```

```
print("Addition =", mymath.addition(a, b))  
print("Subtraction =", mymath.subtraction(a, b))  
print("Multiplication =", mymath.multiplication(a, b))  
print("Division =", mymath.division(a, b))  
print("Modulo =", mymath.modulo(a, b))
```

```
print("Square of first number =", mymath.square(a))  
print("Factorial of first number =", mymath.factorial(a))
```

Practical-25

AIM: Write a program to check whether a given string is palindrome or not.

CODE:

```
def is_palindrome(s):  
    # Remove any spaces and convert the string to lowercase  
    s = s.replace(" ", "").lower()  
  
    # Check if the string is equal to its reverse  
    return s == s[::-1]  
  
# Example usage  
string = input("Enter a string to check if it is a palindrome: ")  
  
if is_palindrome(string):  
    print(f'{string}' is a palindrome.")  
else:  
    print(f'{string}' is not a palindrome.")
```

Practical-26

AIM: Write a program to read a date in the format DD/MM/YYYY and print the same date in MM-DD-YYYY format.

CODE:

```
def convert_date_format(date_str):
    # Split the input date string into day, month, and year
    day, month, year = date_str.split('/')

    # Create the new date string in MM-DD-YYYY format
    new_date_str = f"{month}-{day}-{year}"

    return new_date_str

# Example usage
input_date = input("Enter a date in DD/MM/YYYY format: ")
output_date = convert_date_format(input_date)

print(f"The date in MM-DD-YYYY format is: {output_date}")
```

Practical-27

AIM: Write a python code which joins all items in a tuple into a string, using Hash character as separator.

CODE:

```
def join_tuple_with_hash(tup):  
    # Join all items in the tuple into a string with '#' as separator  
    return '#'.join(map(str, tup))
```

Example usage

```
my_tuple = ('apple', 'banana', 'cherry', 'date')  
result = join_tuple_with_hash(my_tuple)  
print("Joined string:", result)
```

Practical-28

AIM: Write a python code that checks whether given character in string is in uppercase or not.

CODE:

```
def is_uppercase(char):  
    """  
    Check if the given character is in uppercase.  
    """  
    if char.isupper():  
        return True  
    else:  
        return False  
  
# Example usage  
input_string = input("Enter a string: ")  
character_to_check = input("Enter the character to check: ")  
  
if len(character_to_check) != 1:  
    print("Please enter a single character.")  
else:  
    if character_to_check in input_string:  
        if is_uppercase(character_to_check):  
            print(f"The character '{character_to_check}' is in uppercase.")  
        else:  
            print(f"The character '{character_to_check}' is not in uppercase.")  
    else:  
        print(f"The character '{character_to_check}' is not found in the string.")
```

Practical-29

Aim: Write a python code that checks whether given character in string is in lowercase or not.

CODE:

```
string = input("Enter a string: ")
position = int(input("Enter the position of the character: "))

if position >= 0 and position < len(string):
    ch = string[position]

    if ch.islower():
        print("Character", ch, "is in lowercase.")
    else:
        print("Character", ch, "is not in lowercase.")
else:
    print("Invalid position.")
```

Practical-30

AIM: Write a Python code which converts upper case into lower and lower case in uppercase.

CODE:

```
# Python program to convert uppercase to lowercase
# and lowercase to uppercase

string = input("Enter a string: ")

result = string.swapcase()

print("Converted string:", result)
```

Practical-31

AIM: Write a code in python which give right justification.

CODE:

```
# Python program for right justification
```

```
text = input("Enter a string: ")
```

```
width = int(input("Enter total width: "))
```

```
result = text.rjust(width)
```

```
print("Right Justified String:")
```

```
print(result)
```

Practical-32

AIM: Write a Python code which replaces one word with another word.

CODE:

```
# Python program to replace one word with another word
```

```
string = input("Enter a sentence: ")  
old_word = input("Enter the word to replace: ")  
new_word = input("Enter the new word: ")
```

```
result = string.replace(old_word, new_word)
```

```
print("Updated sentence:")  
print(result)
```

Practical-33

AIM: Write a program to create a text file and write a string to it.

CODE:

```
# Python program to create a text file and write a string to it
```

```
file = open("sample.txt", "w")
```

```
text = input("Enter a string: ")
```

```
file.write(text)
```

```
file.close()
```

```
print("String written successfully to sample.txt")
```

Practical-34

AIM: Write a program to Read an entire text file.

CODE:

```
# Python program to read an entire text file
```

```
file = open("sample.txt", "r")
```

```
content = file.read()
```

```
print("Contents of the file:")
```

```
print(content)
```

```
file.close()
```

Practical-35

AIM: Write a program to write a string to a file.

CODE:

```
# Python program to write a string to a file
```

```
file = open("output.txt", "w")
```

```
string = input("Enter a string: ")
```

```
file.write(string)
```

```
file.close()
```

```
print("String written successfully to output.txt")
```

Practical-36

AIM: Write a program to write a list of strings to a file.

CODE:

```
# Python program to write a list of strings to a file
```

```
file = open("strings.txt", "w")
```

```
strings = ["Apple\n", "Banana\n", "Mango\n", "Orange\n"]
```

```
file.writelines(strings)
```

```
file.close()
```

```
print("List of strings written successfully to strings.txt")
```

Practical-37

AIM: Write a program to Count the number of lines, words in a file.

CODE:

```
# Python program to count the number of lines and words in a file
```

```
file = open("sample.txt", "r")
```

```
lines = file.readlines()
```

```
line_count = len(lines)
```

```
word_count = 0
```

```
for line in lines:
```

```
    words = line.split()
```

```
    word_count += len(words)
```

```
file.close()
```

```
print("Number of lines:", line_count)
```

```
print("Number of words:", word_count)
```